

Relatório de Implementação: Simulação 3D de Cubo Mágico

Integrantes: Pedro Ayres, Victor Dantas, João Carlos Pires, Roberto Campos

1. Objetivo do Projeto

Desenvolver uma simulação computacional interativa de um Cubo Mágico (Rubik's Cube) 3x3x3, aplicando conceitos fundamentais de Computação Gráfica, como transformações geométricas (rotação e translação), manipulação de câmera espacial e renderização em tempo real, utilizando a biblioteca Three.js.

2. Metodologia e Tecnologias

O projeto foi construído utilizando o ecossistema Web padrão (HTML5, CSS3 e JavaScript), integrado ao motor gráfico WebGL abstraído pela biblioteca Three.js. A arquitetura do sistema foi dividida de forma modular, separando responsabilidades entre interface, lógica geométrica e ambiente de renderização.

- **index.html** e **style.css**: responsáveis pela estrutura da interface de usuário (UI) e HUD sobreposto ao canvas.
- **cubo.js**: encapsulamento da classe Cubo, responsável pela lógica de negócio, criação das malhas geométricas e matemática das rotações.
- **main.js**: configuração da cena gráfica, câmera, iluminação e loop principal de renderização.

3. Detalhamento Técnico (Implementação)

3.1 Modelagem Geométrica e Materiais

O cubo principal foi construído a partir de 27 subcubos independentes ("cubinhos"), organizados em uma matriz 3x3x3.

- Utilizou-se a primitiva **THREE.BoxGeometry** para instanciar cada malha geométrica.
- O mapeamento de cores seguiu o padrão tradicional do Cubo Mágico, aplicando arrays de **THREE.MeshBasicMaterial** para colorir individualmente cada face.
- Faces internas receberam coloração escura para otimização visual e melhor contraste gráfico.

3.2 Câmera e Iluminação

- **Câmera**: foi utilizada a **PerspectiveCamera** para simular o campo de visão humano. Integrada ao **OrbitControls**, permite inspeção completa do cubo em 360 graus e controle de zoom.
- **Iluminação**: foi implementada uma iluminação híbrida utilizando **AmbientLight** para iluminação global e **DirectionalLight** para gerar contraste direcional nas faces do objeto.

3.3 Transformações Geométricas (Rotação)

Para solucionar o problema de rotacionar apenas determinadas faces do cubo sem comprometer o alinhamento espacial dos demais blocos, adotou-se uma estratégia de agrupamento dinâmico baseada em **THREE.Group**.

O funcionamento ocorre em quatro etapas:

1. O sistema identifica quais cubinhos pertencem à face selecionada.
2. Os cubinhos identificados são removidos temporariamente da cena principal e adicionados a um pivô centralizado (**THREE.Group**).
3. O grupo sofre uma rotação interpolada suavemente através do loop de animação baseado em **requestAnimationFrame**.
4. Após atingir 90 graus ($\pi/2$ radianos), as matrizes globais são atualizadas através de **updateMatrixWorld()**, e os cubinhos retornam à cena principal com suas novas coordenadas espaciais.

Para minimizar erros acumulados por operações de ponto flutuante, foi aplicado um processo de arredondamento matemático nas coordenadas finais após cada rotação.

4. Funcionalidades Interativas

A aplicação possui um ciclo completo de interação em tempo real, acessível tanto por interface gráfica quanto por atalhos de teclado.

- Movimentos independentes das faces Superior (U), Frontal (F) e Inferior (D).
- Sistema de Shuffle baseado em movimentos aleatórios temporizados.
- Sistema de Reset para reconstrução completa da estrutura do cubo.
- Contador dinâmico de movimentos exibido no HUD.
- Controle de zoom utilizando as teclas “+” e “-”.
- Controle orbital da câmera através do mouse com OrbitControls.

5. Dificuldades Encontradas

Durante o desenvolvimento, diversos desafios técnicos relacionados à manipulação espacial em ambiente 3D precisaram ser solucionados.

1. Manter o alinhamento correto dos cubinhos após rotações sucessivas.
2. Impedir novas entradas enquanto uma rotação ainda estava em execução.
3. Gerenciar corretamente grupos temporários e matrizes globais após transformações sucessivas.

6. Divisão de Tarefas

- **Pedro Ayres:** configuração da infraestrutura gráfica, iluminação e OrbitControls.
- **Victor Dantas:** desenvolvimento da classe Cubo e modelagem geométrica.
- **João Carlos Pires:** implementação das transformações geométricas e lógica das rotações.
- **Roberto Campos:** desenvolvimento da interface, eventos, Shuffle/Reset e documentação.

7. Conclusão

O desenvolvimento do projeto atendeu aos requisitos técnicos propostos pela disciplina. A utilização do Three.js permitiu aplicar, na prática, conceitos fundamentais de Computação Gráfica, como transformações geométricas, manipulação matricial, estrutura hierárquica de cena (Scene Graph) e gerenciamento de estados de animação em tempo real.

Além da consolidação dos conceitos teóricos, o projeto proporcionou experiência prática com renderização WebGL, arquitetura modular em JavaScript e interação tridimensional em ambiente web.